

Ai Engineering Chap 8: Software Engineering Principles in Python

Lesson 1: Software Engineering & Data Science

Python, Data Science, Software Engineering

- **Modularity:** You discovered that writing modular code means breaking down a program into smaller, manageable, and reusable pieces. This approach enhances readability and makes maintenance easier. For instance, using packages, classes, and methods in Python helps achieve modularity.
- **Documentation:** The importance of documenting your code was highlighted. Good documentation practices, such as comments, docstrings, and writing self-documenting code, make it easier for others (including your future self) to understand and work with your code.
- **Testing:** You learned that testing is essential for ensuring your code works as expected. Tools like the pytest package can automate testing, helping catch errors early and maintain code quality over time.

Additionally, you practiced writing modular code in Python through an exercise involving the `numpy` package. You used the `numpy` array class and its `sort` method to sort a list of numbers:

```
import numpy as np

arr = np.array([8, 6, 7, 5, 3, 0, 9])
arr.sort()

print(arr)
```

Introduction to Packages and Documentation

- **Python Packages:** These are modules or collections of modules that can be installed from the Python Package Index (PyPI) using `pip`, a package manager for Python. For example, installing the `numpy` package is as simple as running `pip install numpy` in your command line. This process not only installs the requested package but also any dependencies it may have, provided they are available on PyPI.
- **Using `pip`:** You learned that `pip` stands for "Pip Installs Packages" and is a recursive acronym. It's a tool that allows you to install, update, and manage additional libraries and dependencies that are not distributed with the standard Python library.
- **Documentation:** Understanding and leveraging documentation is crucial for effectively using packages. You explored how to access a function's documentation using the `help()` function in Python. For instance, to learn about `numpy`'s `busday_count` method, you would use `help(numpy.busday_count)`. This reveals the function's purpose, parameters, expected output, and even provides examples.

- **Practical Application:** Through exercises, you practiced installing packages with pip and leveraging documentation to understand how to use functions within those packages. This hands-on approach reinforced the lesson's concepts and demonstrated their application in real-world scenarios.

Conventions and PEP8

- **Conventions in Software Engineering:** Just like social conventions that vary across different cultures, software engineering has its own set of conventions to ensure smooth collaboration and understanding among developers.
- **PEP 8 Overview:** PEP 8 provides guidelines on how to format Python code, including recommendations on indentation, whitespace usage, and overall code layout to enhance readability.
- **Tools for Ensuring PEP 8 Compliance:** You explored how to use the `pycodestyle` package to automatically check your code for PEP 8 compliance. This tool acts like a spellchecker for your code, identifying parts that do not adhere to the style guide.

Here's an example of how to use `pycodestyle` to check files for PEP 8 compliance:

```
# Import needed package
import pycodestyle

# Create a StyleGuide instance
style_checker = pycodestyle.StyleGuide()

# Run PEP 8 check on multiple files
result = style_checker.check_files(['nay_pep8.py', 'yay_pep8.py'])

# Print result of PEP 8 style check
print(result.messages)
```

Lesson 2: Writing a Python Module

Lesson 3: Utilizing Classes

Lesson 4: Maintainability